# IO Pset 3 - Aunisha Saha

## Setup

In [19]:
```
import numpy as np
import pandas as pd
```

In [20]:
```
df = pd.read_csv('rust_data_2020.csv')
df = df.sort_values(['bus_id', 'period_id']).reset_index(drop=True)
```

# Part 0

## Q1

In [21]:
```
# get transition probs
df['x_next'] = df.groupby('bus_id')['x_it'].shift(-1)
df['y_next'] = df.groupby('bus_id')['y_it'].shift(-1)

df0 = df[df['y_it'] == 0].copy()
df0 = df0.dropna(subset=['x_next', 'y_next'])

# if the next y is 1
# delta is x_next since mileage resets to 0
df0['delta_x'] = np.where(
    df0['y_next'] == 0,
     df0['x_next'] - df0['x_it'],
    df0['x_next']
).astype(int)

# add tranistion probability for 4
p_delta = (df0['delta_x'].value_counts(normalize=True)
           .reindex([0, 1, 2, 3, 4], fill_value=0)
            .sort_index()
)

print('Transition probabilities: ')
print(p_delta)
print(f'\nSum = {p_delta.sum():.6f}')
```

```
Transition probabilities:
0    0.089072
1    0.437134
2    0.463200
3    0.010594
4    0.000000
Name: delta_x, dtype: float64

Sum = 1.000000
```

## Q2

In [22]:
```
x_states = 201
F = np.zeros((x_states, x_states))

for delta, prob in p_delta.items():
    d = int(delta)
    for x in range(x_states):
        F[x, min(x + d, x_states - 1)] += prob

F = F / F.sum(axis=1, keepdims=True)

print(f'all rows sum to 1: {np.allclose(F.sum(axis=1), 1.0)}')
```

```
all rows sum to 1: True
```

# Part 1

In [23]:
```
import scipy.optimize
from scipy.optimize import minimize

B = 0.975
N = 201
x_vals = np.arange(N)
```

## Q1

In [24]:
```
def compute_ev(theta, tpm, x_vals, beta=B, tol=1e-6):

    a, b, RC = theta

    #quadratic cost function
    u_keep    = -(a * x_vals + b * x_vals**2)
    u_replace = -RC

    EV = np.zeros(len(x_vals))

    while True:
        v_keep    = u_keep + beta * EV
        v_replace = u_replace + beta * EV[0]
        EV_new    = tpm @ np.logaddexp(v_keep, v_replace)
        if np.max(np.abs(EV_new - EV)) < tol:
            return EV_new
        EV = EV_new
```

## Q2

In [25]:
```
def compute_ccp(theta, EV, x_vals, beta=B):

    a, b, RC = theta

    # quadratic cost function
    u_keep = -(a * x_vals + b * x_vals**2)
    u_replace = -RC

    v_keep = u_keep + beta * EV
    v_replace = u_replace + beta * EV[0]
```

```
    # so i dont get overflow warning
    log_p_keep = v_keep - np.logaddexp(v_keep, v_replace)
    return np.exp(log_p_keep)
```

## Q3

In [26]:
```
def nll(theta, x_obs, y_obs, tpm, x_vals):

    EV = compute_ev(theta, tpm, x_vals)

    p_keep = compute_ccp(theta, EV, x_vals)
    p_keep = np.clip(p_keep, 1e-12, 1-1e-12)

    ll = np.where(y_obs==0, np.log(p_keep[x_obs]), np.log(1-p_keep[x_obs]))

    return -np.sum(ll)
```

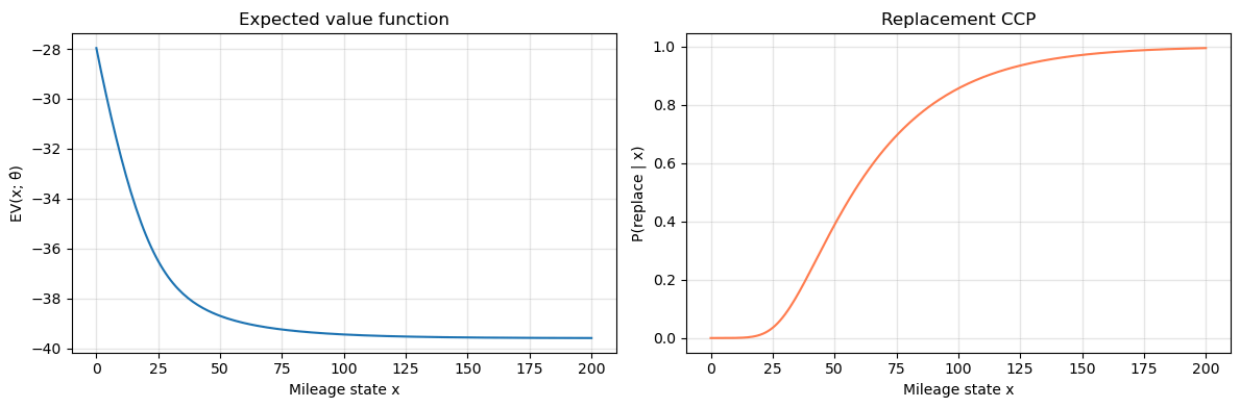
## Q4

In [27]:
```
df["x_lag"] = df.groupby("bus_id")["x_it"].shift(1)

df_model = df.dropna(subset=['x_lag'])


x_obs = df_model["x_lag"].values.astype(int)
y_obs = df_model["y_it"].values.astype(int)


theta0 = np.array([0.1, 0.001, 5.0])

result = minimize(
    fun=nll,
    x0=theta0,
    args=(x_obs, y_obs, F, x_vals),
    method='L-BFGS-B',
    jac=False,
    bounds=[(1e-8, None), (0, None), (1e-6, None)],
    options={'maxiter': 500, 'ftol': 1e-14, 'gtol': 1e-7}
)

a_hat, b_hat, RC_hat = result.x
print(f'a  (linear cost)    = {a_hat:.6f}')
print(f'b  (quadratic cost) = {b_hat:.6f}')
print(f'RC (replacement)    = {RC_hat:.6f}')
print(f'Log-L               = {-result.fun:.4f}')
print(f'Converged           : {result.success}')
```

```
a  (linear cost)    = 0.027033
b  (quadratic cost) = 0.000022
RC (replacement)    = 12.329273
Log-L               = -861.9589
Converged           : True
```

# Graphs for sanity check

In [28]:
```
import matplotlib.pyplot as plt

EV_hat = compute_ev(result.x, F, x_vals)
p_rep  = 1 - compute_ccp(result.x, EV_hat, x_vals)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(x_vals, EV_hat)
axes[0].set_xlabel('Mileage state x')
axes[0].set_ylabel('EV(x; θ̂)')
axes[0].set_title('Expected value function')
axes[0].grid(True, alpha=0.3)

axes[1].plot(x_vals, p_rep, color='coral')
axes[1].set_xlabel('Mileage state x')
axes[1].set_ylabel('P(replace | x)')
axes[1].set_title('Replacement CCP')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




In [ ]: